



OBJECT ORIENTED SYSTEMS (OOS) — 2022

QUESTION 5 COMPLETE DETAILED SOLUTION



Introduction

Question 5 of the 2022 Object Oriented Systems examination belongs to CO4 and focuses entirely on one of the most important areas of software engineering:



Unified Modeling Language (UML)

UML is a standardized graphical language used to:

- Visualize software systems
- Design object-oriented applications
- Document system architecture
- Communicate software structure clearly

In modern software engineering, writing code directly without planning the architecture often creates confusion, poor maintainability, and scalability problems. UML solves this issue by allowing developers to model systems before implementation 🚀.

This question mainly discusses:

- Use Case Diagrams
- Class Diagrams
- Include and Extend Relationships
- Aggregation and Composition
- Multiplicity
- Inheritance Representation

These concepts are extremely important because class diagrams and use case diagrams form the backbone of object-oriented software modeling.



QUESTION 5(a)



Distinguish between include and extend relation of a use case diagram with suitable examples.



Answer

Use Case Diagrams are one of the most fundamental UML diagrams used to represent the functional behavior of a software system from the user's perspective. These diagrams primarily focus on:

- Actors
- Use cases
- Interactions between them

In large systems, many use cases often share common functionalities. Sometimes certain functionalities are compulsory, while others are optional. To represent such relationships effectively, UML provides two important

relationships:

-  **Include Relationship**
-  **Extend Relationship**

Although both establish connections between use cases, their purposes and behavior are completely different



Include Relationship

The **include** relationship is used when one use case compulsorily includes another use case as part of its execution.

In simple words, the included functionality is:

- Common
- Reusable
- Always executed

This relationship helps reduce duplication of repeated functionalities. The notation used is a directed dashed arrow pointing toward the included use case with the stereotype:

```
<<include>>
```

 **Why Include Relationship is Needed?** Suppose multiple use cases require login authentication. Instead of writing login logic repeatedly for:

- Purchase Product
- Check Balance
- Transfer Money

All of them can include **Login**. Thus code duplication and diagram redundancy are reduced .

 **Example of Include Relationship** Consider an Online Banking System. The use case **Transfer Money** must always perform **Authentication**.

```
(Transfer Money) ----<<include>>----> (Authentication)
```

Here authentication is mandatory.

Extend Relationship

The **extend** relationship is used when additional behavior is optional and occurs only under certain conditions. Unlike *include*, *extend* does NOT always execute. It represents conditional or optional functionality. The notation used is:

```
<<extend>>
```

🌟 **Why Extend Relationship is Needed?** Sometimes certain features execute only under special situations. For example:

- Shopping normally proceeds.
- Discount Coupon feature appears *only* if the user has a coupon.

Thus optional behavior can be separated cleanly.

✅ **Example of Extend Relationship** In an Online Shopping System:

```
(Checkout) <-----<<extend>>----- (Apply Discount Coupon)
```

Here discount application is optional. (Note: The arrow points from the extending use case to the base use case).

🌟 Deep Conceptual Difference

The fundamental difference lies in necessity.

- An **include** relationship represents mandatory shared functionality.
- An **extend** relationship represents optional conditional behavior.

This distinction is extremely important in software modeling because it improves modularity, reusability, maintainability, and system clarity.

🌟 Comparison Between Include and Extend

Feature	Include	Extend
Nature	Mandatory	Optional
Purpose	Reuse common functionality	Add conditional behavior
Execution	Always executed	Executed conditionally
Dependency	Base depends on included use case	Base independent of extension
Arrow Direction	Toward included use case	Toward base use case

🌟 **Real-World Intuition** Imagine booking a flight ✈️.

- Whenever flight booking occurs, payment processing is compulsory. Thus payment is **included**.
- However, travel insurance may be optional. Thus insurance **extends** booking.

This real-world analogy makes include and extend relationships much easier to understand 💡.

🎯 Conclusion

Include and extend relationships improve modularity and clarity in UML use case diagrams. Include represents mandatory reusable functionality, whereas extend represents optional conditional behavior. Understanding these relationships is essential for designing scalable and maintainable software systems 🚀.

QUESTION 5(b)

? Show how the various components of a class are represented in a class diagram. What is multiplicity?

 Answer

A **Class Diagram** is one of the most important structural UML diagrams used in Object-Oriented Modeling. It represents:

- Classes
- Attributes
- Methods
- Relationships
- Object interactions

Class diagrams provide the blueprint of an object-oriented system 🧠. Before writing actual code, software engineers use class diagrams to visualize object structure, class responsibilities, and relationships between classes. This helps in designing maintainable and scalable software architecture.

🌟 Structure of a Class Diagram

A class in UML is represented using a rectangle divided into three compartments. These compartments contain:

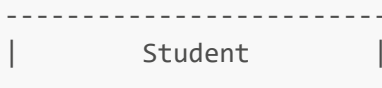
1. Class Name
2. Attributes
3. Methods

🌟 **Representation of Class Components** Suppose we have a class in Java:

```
class Student {
    int roll;
    String name;

    void display() {
    }
}
```

Its UML representation becomes:



```

| - roll : int      |
| - name : String   |
|-----|
| + display() : void |
|-----|

```

☀️ Meaning of Symbols

Symbol	Meaning
+	Public
-	Private
#	Protected

Thus, UML visually represents access specifiers.

☀️ **Attributes in Class Diagram** Attributes represent data members or instance variables. Example: `- roll : int` means:

- Private variable
- Named `roll`
- Datatype `int`

☀️ **Methods in Class Diagram** Methods represent behaviors or functionalities of the class. Example: `+ display() : void` means:

- Public method
- Method name `display`
- Returns `void`

☀️ **Relationships in Class Diagram** Class diagrams also show association, inheritance, aggregation, composition, and dependency. Thus they provide complete structural modeling of the system 🚀.

☀️ What is Multiplicity?

Multiplicity specifies how many objects of one class are associated with another class. It defines the cardinality of relationships and is written near the association lines.

☀️ Common Multiplicity Values

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one
*	Many
1..*	One or more

☀️ **Example of Multiplicity** Suppose: One Department contains many Students. Representation:

```
Department 1 ----- * Student
```

Meaning: One department, multiple students.

🌟 **Importance of Multiplicity** Multiplicity helps software designers understand:

- Object interaction limits
- Relationship constraints
- Database mapping
- System scalability

Without multiplicity, relationships become ambiguous ⚠️.

🌟 **Real-World Intuition** Consider a school:

- One principal manages one school (1 to 1).
- One school contains many students (1 to *).
- One student belongs to one school (* to 1).

Multiplicity models these relationships mathematically.

🎯 **Conclusion**

Class diagrams visually represent the structure of object-oriented systems using classes, attributes, methods, and relationships. Multiplicity further defines the number of objects participating in relationships, making UML diagrams precise and meaningful 🚀.

📄 QUESTION 5(c)

❓ **Distinguish between aggregation and composition relation of a class diagram with proper examples. Show how inheritance is indicated in a class diagram.**

✅ **Answer**

In Object-Oriented Modeling, classes often maintain relationships with one another. UML provides different types of relationships to represent different levels of dependency and ownership.

Among these relationships, two of the most important are:

- 🔥 **Aggregation**
- 🔥 **Composition**

Although both represent "HAS-A" relationships, they differ significantly in ownership strength and object lifetime management 💡.

🌟 Aggregation Relationship

Aggregation represents a **weak** ownership relationship. In aggregation, child objects can exist independently of the parent object. This means destroying the parent does NOT destroy child objects.

Aggregation represents a “HAS-A” relationship with loose coupling. In UML, aggregation is represented using a **Hollow Diamond** (◇).

☀ **Example of Aggregation** Consider: Department and Teacher. A department contains teachers. However:

- Teachers can exist independently.
- Teachers may move to another department.

Thus the relationship is aggregation. Representation:

Department ◇—— Teacher

☀ Composition Relationship

Composition represents a **strong** ownership relationship. In composition, child objects cannot exist independently. If the parent object is destroyed, the child objects are also destroyed automatically.

Composition represents a strong “PART-OF” relationship. In UML, composition is represented using a **Filled Diamond** (◆).

☀ **Example of Composition** Consider: House and Room. Rooms cannot meaningfully exist independently without a house. If the house is destroyed, the rooms are also destroyed. Representation:

House ◆—— Room

☀ Deep Conceptual Difference

- Aggregation indicates sharing or loose association.
- Composition indicates ownership and lifecycle dependency.

This distinction becomes extremely important in memory management, object lifecycle modeling, software architecture, and database mapping.

☀ Comparison Between Aggregation and Composition

Feature	Aggregation	Composition
Ownership	Weak	Strong
Dependency	Independent	Dependent
Child Lifetime	Independent	Dependent on parent
UML Symbol	Hollow Diamond ◇	Filled Diamond ◆
Relationship Type	HAS-A	PART-OF

☀ **Real-World Intuition** Imagine a university and professors 🎓. Professors can exist independently and move between universities. Thus **aggregation** occurs. Now imagine a human body and a heart ❤️. A heart

cannot independently function as a meaningful entity separated from the body. Thus **composition** occurs.

☀ Inheritance in UML Class Diagram

Inheritance represents an **IS-A** relationship. It enables one class to inherit properties and methods of another class. Inheritance promotes code reuse, extensibility, and runtime polymorphism.

☀ **UML Representation of Inheritance** Inheritance is represented using a **solid line with a hollow triangular arrow** pointing toward the parent class.

Example:



Meaning: **Dog** inherits **Animal**.

☀ Example in Java

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog Barking");
    }
}
```

Here, **Dog** inherits **Animal** and can access the methods of **Animal**.

☀ Importance of Inheritance

Inheritance provides:

- Code reusability
- Hierarchical classification
- Easier maintenance
- Extensibility
- Polymorphism

Thus it forms one of the core foundations of Object-Oriented Programming 🚀.

🎯 Conclusion

Aggregation and composition are two important UML relationships representing different ownership strengths. Aggregation models weak dependency, whereas composition models strong lifecycle dependency. Inheritance, represented using hollow triangular arrows, establishes hierarchical relationships and promotes reusable object-oriented design 🚀.